

Optimization in 4D

Daniel Ari Friedman
ORCID: 0000-0001-6232-9096
Email: daniel@activeinference.institute
DOI: 10.5281/zenodo.16887791

October 07, 2026

Contents

1 Optimization in 4D	1
1.1 Overview	1
1.2 Nelder-Mead on Integer Lattice	1
1.2.1 Parameters	1
1.3 Volume-Level Dynamics	2
1.4 Quadray Lattice Optimization Pseudocode	2
1.4.1 Figures	2
1.5 Discrete Lattice Descent (Information-Theoretic Variant)	2
1.6 Convergence and Robustness	6
1.7 Information-Geometric View (Einstein.4D analogy in metric form)	6
1.7.1 Fisher Information as Riemannian Metric	6
1.7.2 4D Framework Integration through Fisher Information	6
1.7.3 Comprehensive Fisher Information Analysis	7
1.7.4 Natural Gradient Descent: Geodesic Motion on Information Manifold	7
1.7.5 Quadray-Specific Considerations	10
1.7.6 Variational Free Energy and Active Inference Integration	10
1.8 Multi-Objective and Higher-Dimensional Notes	10
1.9 External Validation and Computational Context	11
1.10 Results	11

1 Optimization in 4D

1.1 Overview

This section describes optimization methods adapted to the integer Quadray lattice: a discrete Nelder-Mead simplex method with lattice projection, a greedy 12-neighbor descent, and Fisher-metric (natural-gradient) guidance for continuous parameter spaces. The integer lattice supplies natural quantization of both steps and simplex volumes, which the convergence criteria below exploit; higher-dimensional extensions are developed in Section 5 (`05_extensions.md`).

1.2 Nelder-Mead on Integer Lattice

- **Adaptation:** the four standard simplex operations — reflection (α), expansion (γ), contraction (ρ), and shrink (σ) — applied to a simplex of four lattice points, with the objective f evaluated on the embedded coordinates (x, y, z) of each vertex.

- **Projection:** candidate moves are formed with per-component integer truncation of the scaled offsets (the centroid of the best three uses floor division), then mapped back to the canonical lattice representative by projective normalization, so the entire trajectory stays on the lattice.
- **Volume tracking:** monitor the exact IVM tetravolume (absolute quadray determinant divided by 4, kept as an exact Fraction; Section 3) as a convergence diagnostic; discrete steps create stable volume plateaus.
- **Degenerate-simplex restart:** a zero-volume (collinear/coplanar) simplex cannot leave its own affine subspace, since every NM move is an affine combination of the vertices. When the simplex collapses (zero volume with spread below tolerance), the algorithm probes ± 1 and ± 2 lattice steps along each quadray axis from the best vertex; if any probe improves the best value, it re-seeds a full-volume simplex along the three most promising axes (a CVP-style restart, consuming one iteration), otherwise the collapse is accepted as convergence and the run terminates.

1.2.1 Parameters

- **Reflection** $\alpha \approx 1$
- **Expansion** $\gamma \approx 2$
- **Contraction** $\rho \approx 0.5$
- **Shrink** $\sigma \approx 0.5$

References: original Nelder-Mead method and common parameterizations in optimization texts and survey articles; see overview: [Nelder-Mead method](#).

1.3 Volume-Level Dynamics

- Simplex tetravolume decreases in discrete steps ($|\det|/4$ values such as 0.5, 0.25, 0 in IVM units), producing plateaus (“energy levels”) between accepted moves.
- Termination: a collapsed simplex (zero volume) with objective spread below tolerance τ is checked by axis probes (± 1 , ± 2 lattice steps along each quadray axis from the best vertex); if no probe improves the best value, the run terminates, and if one does, a CVP-style restart re-seeds a full-volume simplex along the most promising axes. A degenerate simplex (zero volume) with spread still above τ simply continues — its affine moves remain confined to the collapse subspace while the spread shrinks toward collapse.
- Monitoring: record best/worst objective, spread, and exact IVM volume at each iteration (see `simplex_trace.png` below).

1.4 Quadray Lattice Optimization Pseudocode

```

1 while not converged:
2     order vertices by objective
3     centroid of best three
4     propose reflected (then possibly expanded/contracted) point
5     accept per standard tests; else shrink toward best
6     if simplex volume is zero and spread is below tolerance:
7         probe +/-1 and +/-2 lattice steps along each quadray axis from best
8         if any probe improves best: restart (full-volume simplex along 3 best axes); else stop
9     update integer volume and function spread trackers

```

1.4.1 Figures

The 2×2 panel below shows the simplex itself at iterations 0, 3, 6, and 9; the plateau structure of the trace above appears as iterations where the simplex repositions without improving its best value.

Raw artifacts: the full trajectory animation `simplex_animation.mp4` and per-frame vertices (`simplex_animation_vertices.csv/.npz`) are available in `quadmath/output/`.

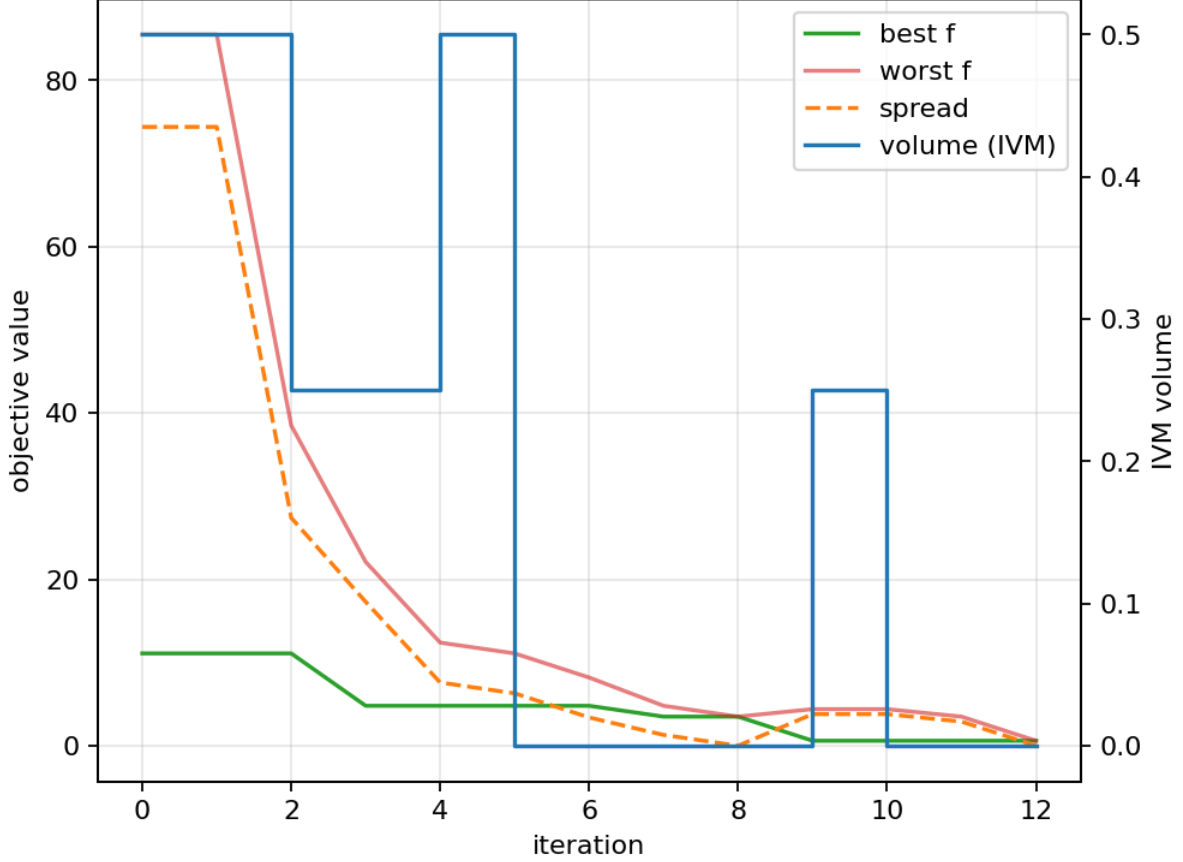


Figure 1: **Per-iteration diagnostics for discrete Nelder-Mead on the integer Quadray lattice.** Time series of the 13 recorded states (iterations 0–12, i.e. 12 optimization iterations after the initial simplex), plotted by `plot_simplex_trace` (`src/quadmath/viz/visualize.py`) from the run in `quadmath/scripts/simplex_animation.py` for the penalized objective $f(q) = (x-2)^2 + (y-2)^2 + (z-2)^2 + 0.1|x+y+z|$, plus +5 in the quadrant $x < 0 \wedge y < 0$ and +10 when any of $|x|, |y|, |z|$ exceeds 4, evaluated at the embedded Cartesian coordinates (x, y, z) from `to_xyz` with `DEFAULT_EMBEDDING`. **Left axis** (dimensionless objective units): best value $\min_i f(v_i)$ (green), worst value $\max_i f(v_i)$ (faint red), and spread $\max_i f(v_i) - \min_i f(v_i)$ (orange, dashed); only the best value is monotone — the restart at iteration 9 raises the worst value from 3.5 to 4.4. **Right axis** (blue step line): exact tetravolume in IVM units ($|\det|/4$, i.e. 0.5, 0.25, 0). The best value descends in plateaus (11.1 on 0–2, 4.8 on 3–6, 3.5 on 7–8, 0.6 on 9–12); the spread reaches zero at iterations 8 and 12, and the iteration-8 collapse triggers the CVP-style restart visible at iteration 9 (spread 0 $\rightarrow$ 3.8, volume 0 $\rightarrow$ 0.25). Regenerate with `uv run python quadmath/scripts/simplex_animation.py`; raw data in `quadmath/output/data/simplex_trace.csv/.npz`.

Nelder-Mead Simplex Evolution on Integer Quadray Lattice

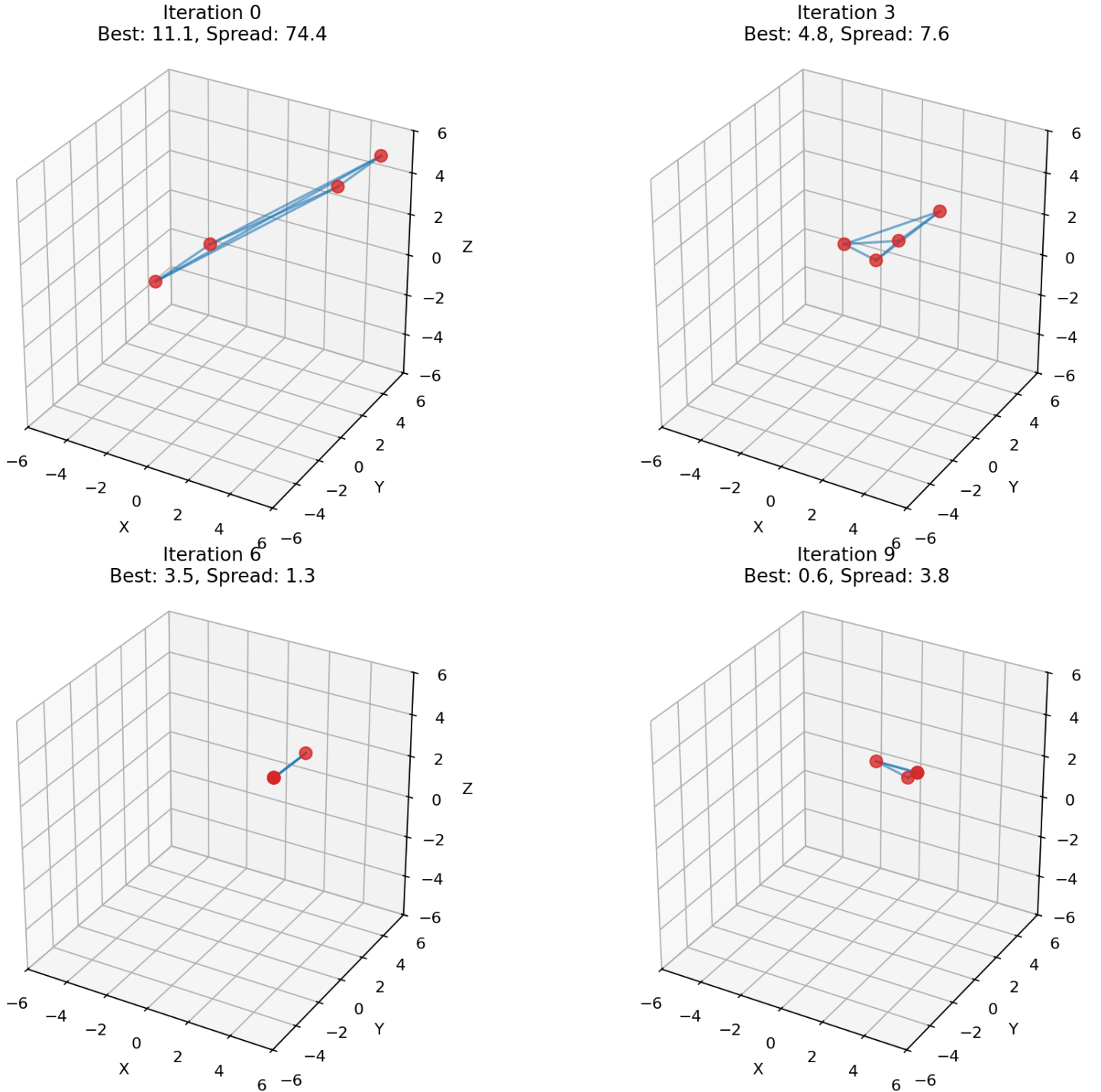


Figure 2: **Nelder-Mead simplex evolution on the integer Quadray lattice (2×2 panel).** The four-vertex simplex (vertices as red spheres, the six tetrahedral edges as blue lines) at iterations 0, 3, 6, and 9 of the same run as the trace figure, plotted in the embedded (x, y, z) space produced by `to_xyz` with `DEFAULT_EMBEDDING`; each axis spans $[-6, 6]$, and each panel title reports that iteration's best objective value and spread. **Top-left (iteration 0):** the widely dispersed initial simplex (`Quadray(5,0,0,0)`, `Quadray(4,1,0,0)`, `Quadray(0,4,1,0)`, `Quadray(1,1,1,0)`). **Top-right (iteration 3):** contraction toward the main basin. **Bottom-left (iteration 6):** near-collapsed simplex (spread 1.3) just before the degenerate restart. **Bottom-right (iteration 9):** the re-seeded simplex contracting again (spread 3.8); the run terminates at iteration 11 with all four vertices coinciding at `Quadray(2,0,0,0)`, i.e. embedded $(2, 2, 2)$. Generated by `quadmath/scripts/simplex_animation.py`.

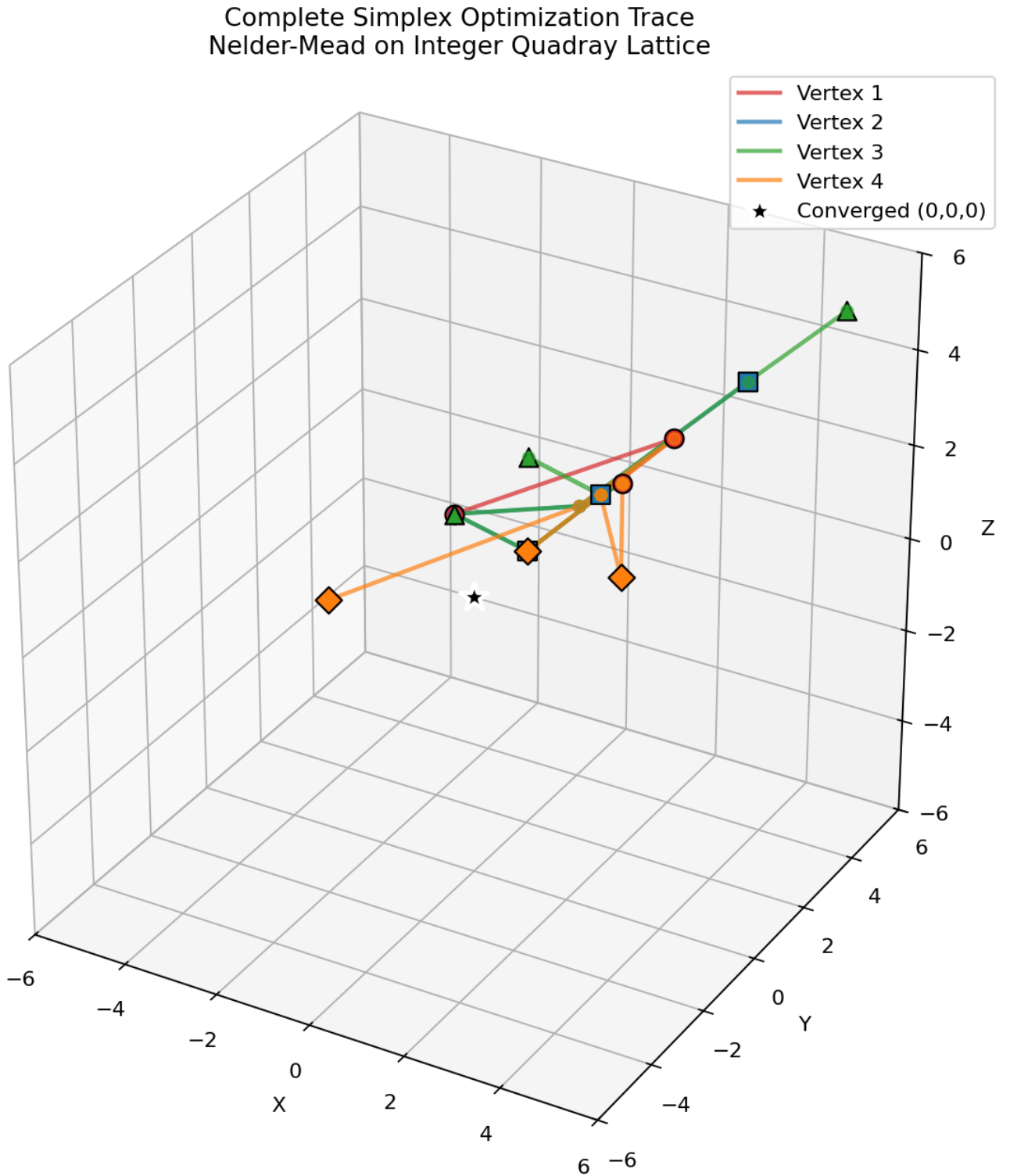


Figure 3: **Complete simplex vertex trajectories (3D)**. Full paths of the four simplex vertices across all 12 recorded iterations of the same run, in the same embedded (x, y, z) axes $([-6, 6]$ per direction) as the panel above. Each vertex trace uses a fixed color and marker (red circle, blue square, green triangle, orange diamond); large black-edged markers flag iterations 0, 4, and 8. The traces show coordinated contraction toward the converged point $(2, 2, 2)$, with the degenerate-simplex restart visible as the outward jump near iteration 8. The script also draws a black star labeled “Converged (0,0,0)” at the embedded origin; the vertices themselves converge to $(2, 2, 2)$, so read the star as a decorative marker, not the optimum. Generated by `quadmath/scripts/simplex_animation.py`.

1.5 Discrete Lattice Descent (Information-Theoretic Variant)

- Integer-valued greedy descent over the IVM: from the current lattice point q , evaluate f at the 12 nearest neighbors (the distinct permutations of $(2, 1, 1, 0)$ in quadray offsets), then move to the minimizing neighbor.
- Each accepted move strictly decreases f , so the objective is monotone along the path; the walk terminates at a local minimum of f over the IVM adjacency, i.e. when no neighbor improves on the current value.
- The objective may be geometric (e.g. Euclidean distance in an embedding) or information-theoretic (e.g. a local free-energy proxy).
- API: `discrete_ivm_descent` in `src/quadmath/optimize/discrete_variational.py`. Animation helper: `animate_discrete_path` in `src/quadmath/viz/visualize.py`.

Short snippet (paper reproducibility):

```
1 from quadmath.core.quadray import Quadray, DEFAULT_EMBEDDING, to_xyz
2 from quadmath.optimize.discrete_variational import discrete_ivm_descent
3 from quadmath.viz.visualize import animate_discrete_path
4
5 def f(q: Quadray) -> float:
6     x, y, z = to_xyz(q, DEFAULT_EMBEDDING)
7     return (x - 0.5)**2 + (y + 0.2)**2 + (z - 0.1)**2
8
9 path = discrete_ivm_descent(f, Quadray(6,0,0,0))
10 animate_discrete_path(path)
```

1.6 Convergence and Robustness

- Discrete steps reduce numerical drift; improved stability vs. unconstrained Cartesian.
- Natural regularization from volume quantization; fewer wasted evaluations.
- Compatible with Gauss-Newton/Natural Gradient guidance using FIM for metric-aware steps (Amari, natural gradient).

1.7 Information-Geometric View (Einstein.4D analogy in metric form)

The Fisher Information Matrix (FIM) provides a fundamental bridge between the three 4D frameworks, establishing a Riemannian metric on parameter space that guides optimization through information geometry. This section demonstrates how the FIM connects Coxeter.4D (Euclidean parameter space), Einstein.4D (information-geometric flows), and Fuller.4D (tetrahedral structure) in a unified optimization framework.

1.7.1 Fisher Information as Riemannian Metric

The empirical Fisher Information Matrix F_{ij} quantifies the local curvature of the log-likelihood surface around parameter estimates, providing a natural metric for parameter space geometry. This fundamental concept in information geometry establishes a Riemannian structure on the statistical manifold, where distances and angles are measured according to the intrinsic geometry of the probability distributions rather than the extrinsic Euclidean geometry of the parameter space.

For a model with parameter vector θ and per-sample scores $\partial_{\theta_i} \log p(x_n; \theta)$, the empirical FIM is the average outer product of score functions, $F_{i,j} = \frac{1}{N} \sum_{n=1}^N \partial_{\theta_i} \log p(x_n; \theta) \partial_{\theta_j} \log p(x_n; \theta)$ (Eq. (??) in the equations appendix). Diagonal entries quantify parameter sensitivity and off-diagonal entries pairwise parameter interactions. The running example below keeps the parameter names $\mathbf{w} = (w_0, w_1, w_2)$ used by `information_demo.py`; the two notations coincide.

The Fisher Information Matrix serves as the natural metric tensor $g_{ij} = F_{ij}$ on the statistical manifold, replacing the Euclidean metric δ_{ij} with a data-dependent metric that reflects the actual curvature structure of the objective function. This geometric interpretation enables the application of differential geometry concepts to

optimization problems, where geodesics (locally distance-minimizing paths) follow the natural gradient direction $F^{-1}\nabla L$ rather than the standard gradient ∇L .

The theoretical foundation of this approach stems from the work of Rao (1945) and Amari (1985), who established information geometry as a framework for analyzing statistical models through differential geometry. The FIM naturally arises as the Hessian of the Kullback-Leibler divergence between nearby probability distributions, making it the canonical choice for measuring distances on the statistical manifold.

In the context of optimization, the FIM provides several key advantages:

1. **Invariance to parameterization:** The natural gradient $F^{-1}\nabla L$ is invariant to smooth, invertible parameter transformations, unlike the standard gradient which depends on the choice of coordinate system.
2. **Optimal step sizing:** The FIM automatically determines appropriate step sizes in different parameter directions, scaling updates according to local curvature.
3. **Geometric consistency:** Optimization follows geodesics on the statistical manifold, respecting the intrinsic geometry of the parameter space rather than imposing an artificial Euclidean structure.

This geometric approach to optimization is particularly powerful in the context of the 4D frameworks, where it provides a unified mathematical language for describing optimization dynamics across different geometric paradigms.

1.7.2 4D Framework Integration through Fisher Information

Coxeter.4D (Euclidean): In standard Euclidean parameter space, the metric tensor is simply δ_{ij} , providing uniform scaling in all directions. The FIM F_{ij} generalizes this to capture the actual curvature structure of the objective function.

Einstein.4D (Minkowski analogy): the Fisher metric $g_{ij} = F_{ij}$ replaces the flat metric, and geodesic motion on the statistical manifold corresponds to the natural gradient update $\theta \leftarrow \theta - \eta F(\theta)^{-1}\nabla_{\theta}L(\theta)$ (Eq. (??) in the equations appendix) — steepest descent measured in the Fisher metric rather than straight-line motion in parameter space.

Fuller.4D (Synergetics): The tetrahedral structure of Quadray coordinates naturally encodes the four-fold partition of optimization problems, while the FIM provides the metric structure for efficient navigation through this space. The discrete nature of the IVM lattice creates natural quantization effects that can be exploited for computational efficiency.

1.7.3 Comprehensive Fisher Information Analysis

Two figures summarize the empirical Fisher analysis of the regression example introduced below: the matrix structure and its eigenspectrum. Both are generated by `quadmath/scripts/information_demo.py` and interpreted through the three 4D frameworks.

Note on the estimator: the matrix in the figure above is computed from per-sample **squared-loss** gradients (`2*x_i*r_i`) at a misspecified `w_est` (`information_demo.py`), i.e. a Gauss-Newton/Gram surrogate. It coincides with the empirical FIM of Eq. (??) only for true score functions — exactly at w_{true} in expectation under Gaussian noise — so treat the displayed matrix as a curvature-scale illustration rather than a model FIM estimate.

1.7.4 Natural Gradient Descent: Geodesic Motion on Information Manifold

The Fisher Information Matrix enables natural gradient descent, which implements geodesic motion on the information manifold. Unlike standard gradient descent that follows straight lines in parameter space, natural gradient descent follows curved paths that respect the intrinsic geometry defined by the FIM.

The natural gradient update used throughout this manuscript is Eq. (??) in the equations appendix, $\theta \leftarrow \theta - \eta F(\theta)^{-1}\nabla_{\theta}L(\theta)$, where η is the step size, F the empirical FIM of Eq. (??), and $\nabla_{\theta}L$ the gradient of the loss. Because F is the metric tensor $g_{ij} = F_{ij}$ of the statistical manifold, this update is steepest descent measured in the Fisher metric — geodesic motion rather than straight-line motion in parameter space.

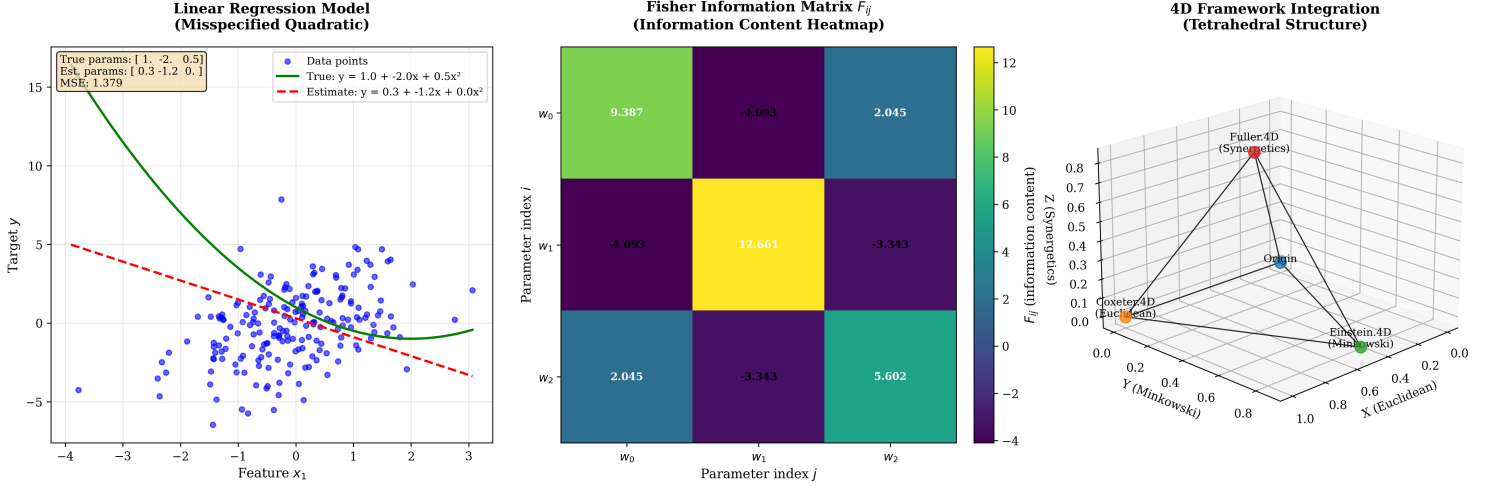


Figure 4: **Empirical Fisher Information Matrix with 4D framework context** (three panels; generated by quadmath/scripts/information_demo.py with seed default_rng(0)). The model is a 3-parameter linear regression $y = \mathbf{x}^\top \mathbf{w} + \varepsilon$ on $N = 200$ samples with standard-Gaussian features and noise $\varepsilon \sim \mathcal{N}(0, 0.1^2)$, generated at $\mathbf{w}_{\text{true}} = (1.0, -2.0, 0.5)$ and evaluated at the deliberately misspecified estimate $\mathbf{w}_{\text{est}} = (0.3, -1.2, 0.0)$. **Left panel:** the data points with the 1-D slice $y(x) = w_0 + w_1x + w_2x^2$ of both parameter vectors (green solid: true; red dashed: estimate) and the MSE annotation. **Center panel:** the 3×3 matrix of per-sample squared-loss gradients $2x_{ni}r_n$ (a Gauss-Newton/Gram surrogate of the FIM; see the estimator note below) as an annotated heatmap, with diagonal entries $\approx (9.39, 12.66, 5.60)$ and off-diagonal entries $\approx (-4.09, 2.05, -3.34)$; units are squared-loss gradient products. **Right panel:** schematic tetrahedron relating Coxeter.4D (Euclidean parameter space, metric δ_{ij}), Einstein.4D (Fisher metric replacing the flat metric), and Fuller.4D (tetrahedral/IVM structure).

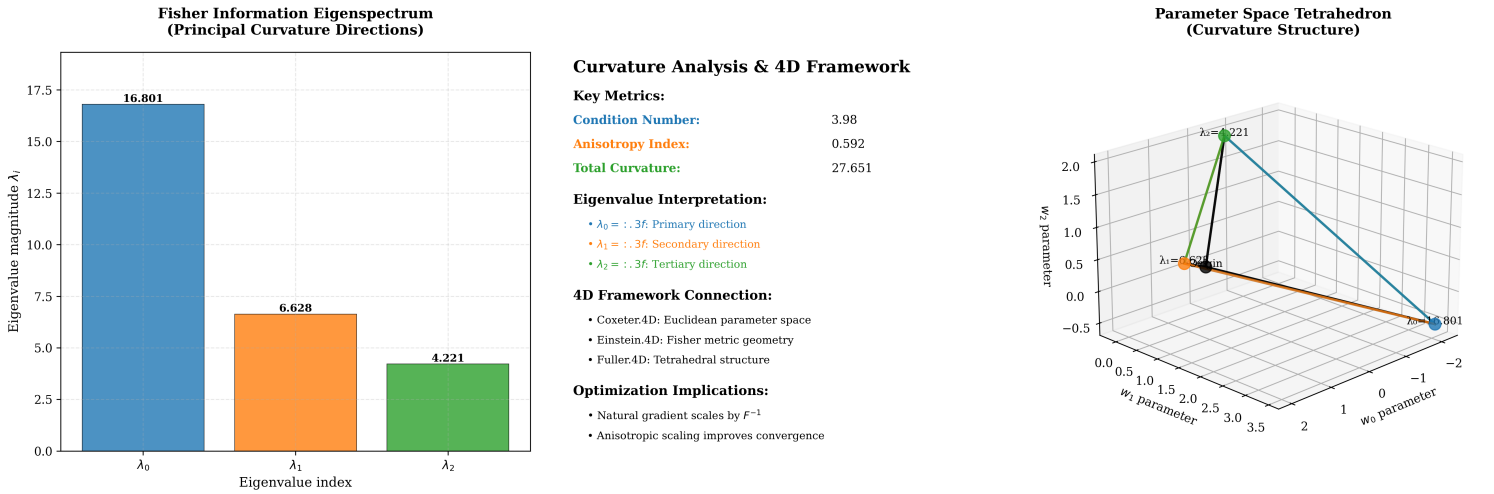


Figure 5: **Fisher Information eigenspectrum and parameter-space curvature** (three panels; generated by quadmath/scripts/information_demo.py from the same seeded run as the matrix figure above). **Left panel:** bar chart of the eigenvalues of the empirical FIM, sorted descending and annotated: $\lambda_0 \approx 16.80$, $\lambda_1 \approx 6.63$, $\lambda_2 \approx 4.22$ (units: squared-loss gradient products) — the principal curvature scales of the loss surface. **Center panel:** text summary of the curvature metrics: condition number $\lambda_{\text{max}}/\lambda_{\text{min}} \approx 3.98$ (anisotropy), anisotropy index ≈ 0.59 , and total curvature (trace of F) ≈ 27.65 , with per-direction and 4D-framework interpretation. **Right panel:** a parameter-space tetrahedron with one vertex at the origin and three vertices along the eigenvector directions scaled by $\sqrt{\lambda_i}$, so the tetrahedron's shape encodes the anisotropy; edge colors mark eigenvalue rank, and vertices are labeled with their eigenvalues. Large eigenvalues mark directions of rapid objective change (where the natural gradient takes small steps); small eigenvalues mark flat directions (larger steps).

The theoretical foundation of natural gradient descent was established by [Amari \(1998\)](#) in the context of information geometry. The key insight is that the natural gradient $F^{-1}\nabla L$ is the steepest descent direction when distances are measured using the Fisher metric rather than the Euclidean metric. This makes natural gradient descent invariant to smooth, invertible parameter transformations, a property that standard gradient descent lacks.

In the context of the 4D frameworks, natural gradient descent provides a unified approach to optimization that respects the intrinsic geometry of each framework:

- **Coxeter.4D:** The natural gradient respects the actual curvature structure of the objective function rather than imposing artificial Euclidean geometry.
- **Einstein.4D:** The Fisher metric replaces the spacetime metric, creating geodesic flows that follow the intrinsic geometry of the parameter space.
- **Fuller.4D:** The tetrahedral structure provides natural coordinate systems where the FIM can exhibit beneficial structural properties.

The efficiency of natural gradient descent comes from its ability to automatically adapt step sizes to local curvature. In directions of high curvature (large eigenvalues of F), the natural gradient takes smaller steps, while in directions of low curvature (small eigenvalues), it takes larger steps. This anisotropic scaling leads to faster convergence and better numerical stability compared to standard gradient descent.

1.7.5 Quadray-Specific Considerations

Under Quadray parameterizations, the FIM often exhibits block-structured and symmetric patterns that simplify matrix inversion for natural-gradient steps. This structural regularity arises from the tetrahedral symmetry of the IVM lattice and can be exploited for computational efficiency.

The discrete nature of the IVM lattice also influences the FIM structure, as parameter updates are constrained to integer coordinate positions. This creates a natural regularization effect that can improve optimization stability and convergence.

1.7.6 Variational Free Energy and Active Inference Integration

The Fisher Information framework naturally extends to variational inference and active inference, where the free energy principle guides both perception and action through information-geometric optimization.

For the full Active Inference treatment — expected free energy, perception and action updates, and further 4D natural-gradient visualizations — see [Section 9: Free Energy and Active Inference](#).

1.8 Multi-Objective and Higher-Dimensional Notes

The extension of these ideas — simplex faces as Pareto trade-off surfaces, integer volume as a solution-diversity measure, and higher-simplex volume decompositions — is developed in [Section 5 \(05_extensions.md\)](#).

1.9 External Validation and Computational Context

The methods above complement the computational framework in Kirby Uerner’s [4dsolutions ecosystem](#); implementation and educational context are collected in the [Resources](#) section.

1.10 Results

On the penalized quadratic of `quadmath/scripts/simplex_animation.py`, the discrete Nelder-Mead reaches the lattice point `Quadray(2,0,0,0)` (embedded $(2,2,2)$, best objective 0.6) in 12 recorded iterations, with the best value descending through the plateaus $11.1 \rightarrow 4.8 \rightarrow 3.5 \rightarrow 0.6$ and one degenerate-simplex restart mid-run; see the simplex figures above and the artifacts in `quadmath/output/`.

Natural Gradient Trajectory (Geodesic Motion on Information Manifold)

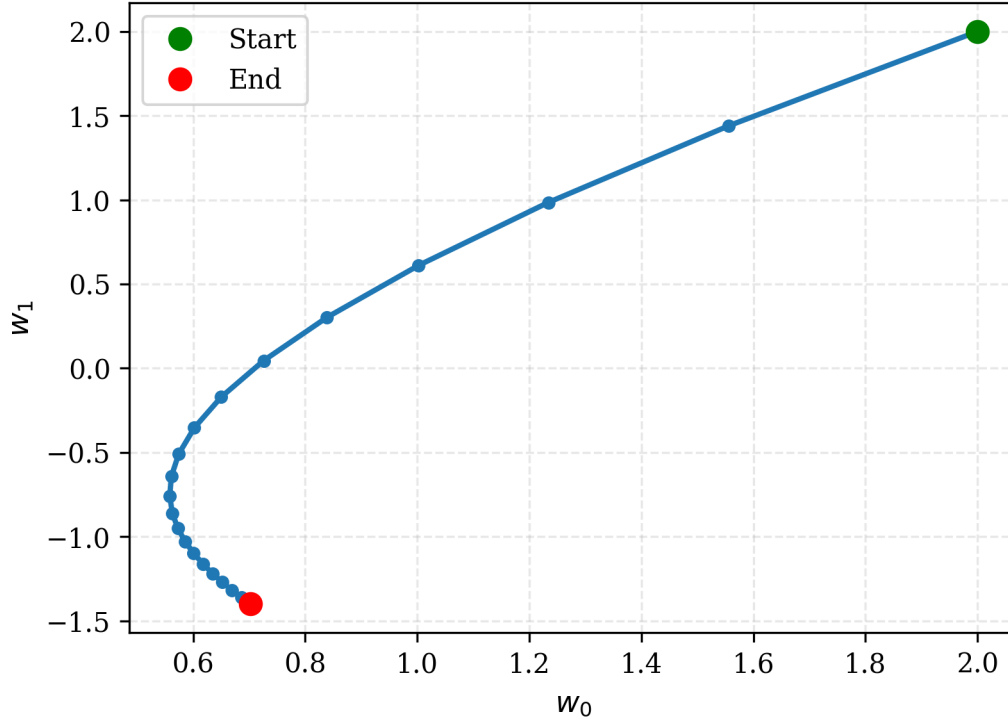


Figure 6: **Natural gradient trajectory on a quadratic bowl** (generated by `quadmath/scripts/information_demo.py`). The objective is the quadratic $L(\mathbf{w}) = \frac{1}{2}(\mathbf{w} - \mathbf{w}_{\text{true}})^\top A (\mathbf{w} - \mathbf{w}_{\text{true}})$ with $\mathbf{w}_{\text{true}} = (1, -2, 0.5)$ and A the positive-definite matrix with diagonal $(3, 2, 1)$ and off-diagonal $A_{12} = 0.5$; the metric used for the steps is the empirical FIM F of the regression example above (plus a 10^{-3} ridge for invertibility), not A . Starting from $(2, 2, 2)$, the blue line with markers shows 20 updates of the form $\Delta \mathbf{w} = -0.5 F^{-1} A (\mathbf{w} - \mathbf{w}_{\text{true}})$ — the 3-parameter trajectory (w_0, w_1, w_2) projected onto the (w_0, w_1) plane (parameter units). Green circle: start; red circle: the final iterate $(0.70, -1.40)$ after 20 steps — a fixed-metric preconditioned run stopped at its step budget, not yet at $\mathbf{w}_{\text{true}} = (1, -2, 0.5)$. The anisotropic F^{-1} preconditioning is visible as unequal progress along the two plotted coordinates.

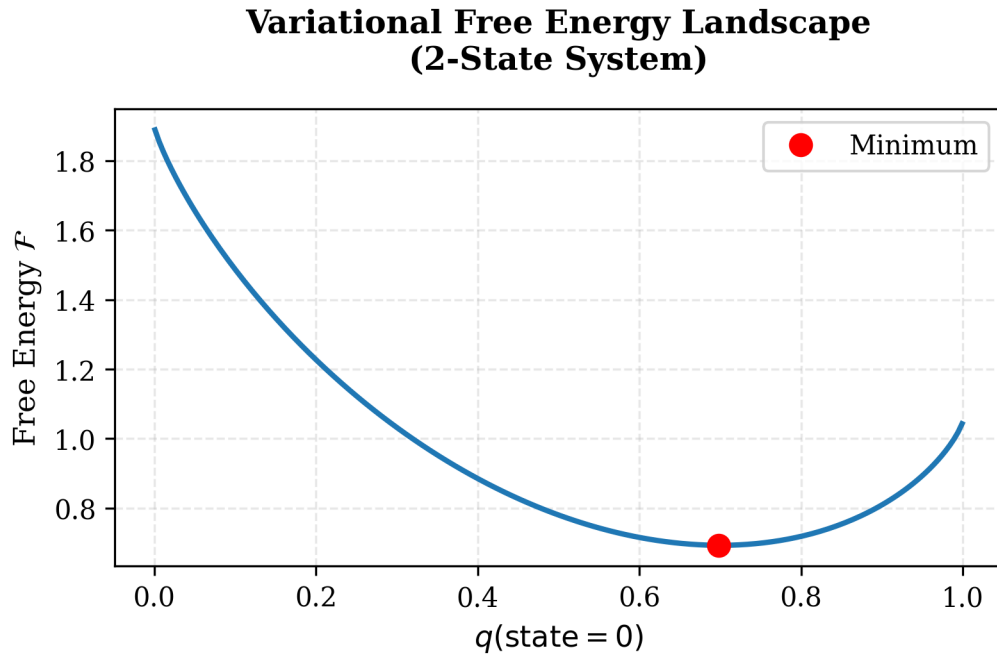


Figure 7: **Variational free energy for a 2-state toy model** (generated by `quadmath/scripts/information_demo.py`). The curve shows the free energy of Eq. (??), $\mathcal{F} = -\log P(o | s) + \text{KL}[Q(s) \| P(s)]$, as a function of the variational parameter $q = Q(\text{state} = 0)$ over the grid $q \in (0, 1)$ (200 points), for likelihood $P(o | s) = (0.7, 0.3)$, uniform prior $P(s) = (0.5, 0.5)$, and variational family $Q(s) = (q, 1 - q)$. **X-axis:** q (dimensionless probability). **Y-axis:** $\mathcal{F}$ in nats. The minimum (red marker) is $\mathcal{F} \approx \ln 2 \approx 0.693$ at $q = 0.7$, where Q coincides with the likelihood — the standard variational result that the optimal Q in the unconstrained family is the posterior itself. In the 4D reading, minimizing $\mathcal{F}$ is geodesic motion under the Fisher metric on the variational manifold (Einstein.4D analogy).